FSTT Local AI Assistant Final Project Report

Module 1: Agentic AI

Student : Chaimae Boulakhrif

Date : 22/10/2026

List of Figures

Figure 1: 01_Installation 4
Figure 2: 04_Python 4
Figure 3: Response of model while internet connection is disabled 5
Figure 4:02_ Models 5
Figure 5:Comparison of the two local models. 7
Figure 6: 03_Chatbot 7
Figure 7: response of the model for conversation history 7
Figure 8 : 05_RAG 9
Figure 9:Results of multimodal LLM 9
Figure 10: 06_Webapp 10
Figure 11:check list of delivrable 13

List of Tables

Table 1: Data of two local models 6

22/10/2026

Contents :

Project overview 3
Part 1: Understanding Local AI 3
Part 2: Installing Ollama 4
Part 3: First Local LLM 5
Part 4: Prompt Engineering 5
Part 5: Model Management 5
Part 6: Comparing Two Local Models 6
Part 7: Building a Local AI Chatbot 7
Part 8: Conversation Memory 7
Part 9: Local AI for Software Engineering 8
Part 10: Structured Output 8
Part 11: Local Document Assistant 8
Part 12: Mini-RAG 8
Part 13: Multimodal AI 9
Part 14: Final Flask Application 10
Part 15: Security Risks 10
Part 16: Final Questions 10
Part 17: Final Mini Project 12
Part 18: Deliverables 12
Conclusion 14

Project overview

This project aimed to build a fully local AI assistant using Ollama, Python, Flask, and local course documents.
The assistant runs on the local computer and provides the following functions:

- answering questions about the supplied course;
- maintaining conversation history;
- retrieving information from local documents;
- producing structured responses for selected questions;
- displaying the answer in a web interface;
- recording response time and generation speed.

The notebook is organized into model testing, prompt engineering, document grounding, conversation memory, structured output, Mini-RAG, Flask development, benchmarking, and final packaging.

Part 1: Understanding Local AI

1. **Definition of Local AI:**

Local AI is artificial intelligence that runs directly on the user's computer instead of sending prompts to a remote cloud service.
In this project, Ollama runs the language model locally. Therefore, prompts, local documents, and generated answers can remain on the user computer.

2. **Local AI compared with Cloud AI:**

A cloud LLM runs on remote servers and is accessed through the Internet or an online API. A local LLM runs on the user's own computer.
Cloud models often provide more computing power and may support larger models. Local models provide better privacy, offline access, direct control over data, and no per-request cloud cost.

3. **Advantages and limitations of local AI:**

✓ **Advantages**

- Better privacy because prompts and documents can remain on the computer.
- Operation without an Internet connection after installation.
- No cloud API cost per token.
- Direct control over the selected model and data.
- Easy integration with Python applications.

✓ **Limitations**

- Performance depends on the local hardware.
- Larger models require more RAM or VRAM.
- Local models may be less capable than the best cloud models.
- The user is responsible for installation, updates, and security.
- CPU inference can be slow on computers with limited resources.

Part 2: Installing Ollama

1. Role of Ollama

Ollama is the local runtime used in this project. It downloads, manages, loads, and serves language models locally. It also provides an HTTP API that Python and Flask can use to send prompts and receive generated answers. The local API is normally available at http://localhost:11434/api.

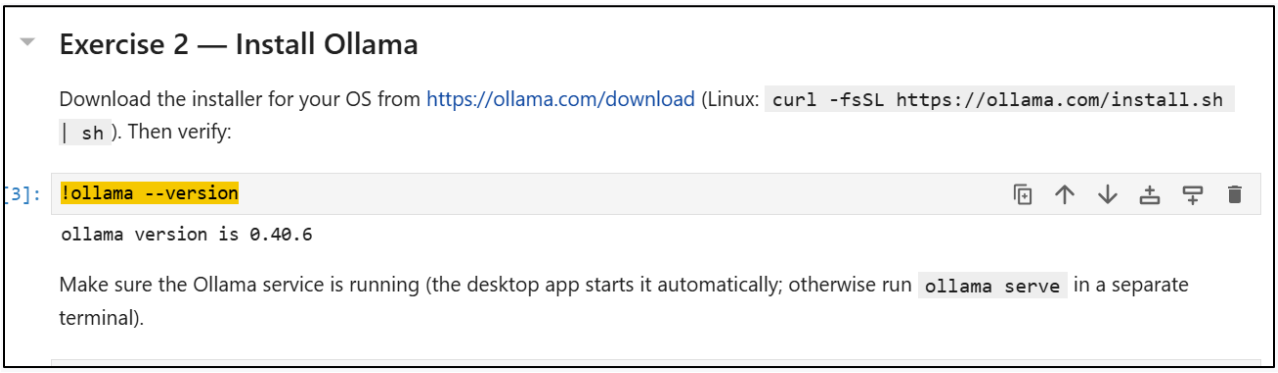


Figure 1: 01_Installation

2. Is the model communicating with a cloud API?

No. The Python program communicates with the local Ollama API running at http://localhost:11434. The prompt is not sent to a cloud API for inference.

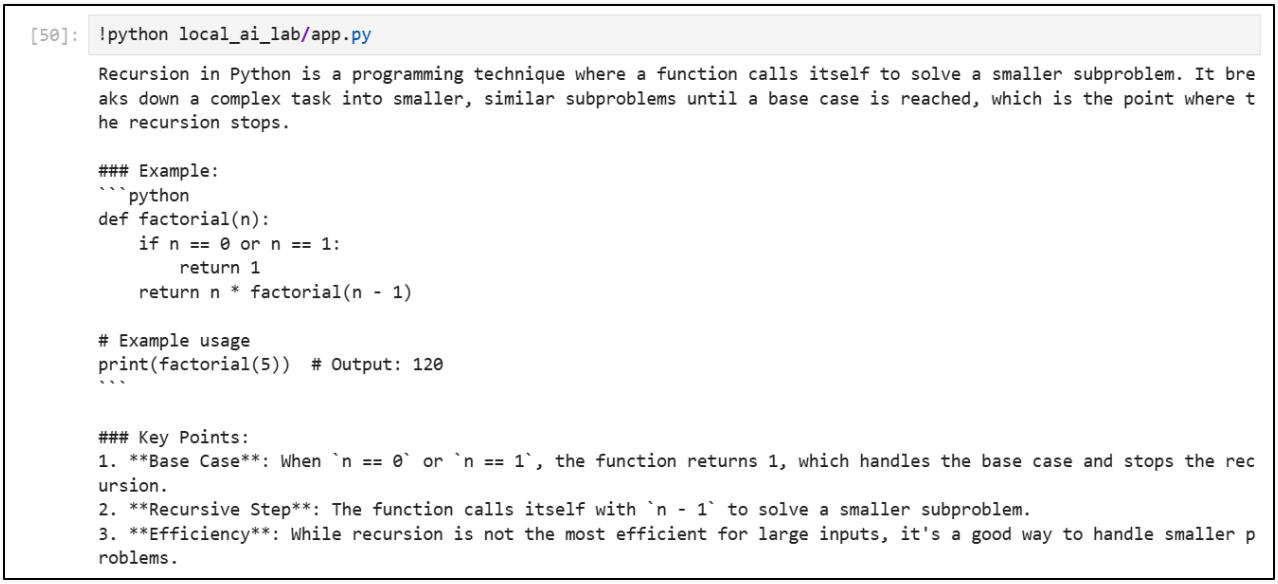


Figure 2: 04_Python

3. Where is inference being performed?

Inference is performed locally on the computer by Ollama using the downloaded qwen3:4b model. In the test, the model ran on the CPU, as indicated by ollama ps, which showed 100% CPU usage.

4. What happens if the Internet connection is disabled?

After disabling the Wi-Fi connection, the model still generated a response in 12.1 seconds. This confirms that inference was performed locally. Internet access is required to download models, updates, or software, but it is not required for generation once Ollama and the model are installed.

```
ask("What is artificial intelligence? Answer in two sentences.")

[12.1s]
'Artificial intelligence refers to the ability of machines to perform tasks that typically require human intelligence, such as learning,
problem-solving, and decision-making. It involves complex systems that analyze data, learn patterns, and make decisions based on that
information.  \n\nIn essence, AI is the intersection of algorithms, data, and learning, enabling machines to execute tasks requiring human-
like reasoning.'
```

Figure 3: Response of model while internet connection is disabled

Part 3: First Local LLM

1. **Models used in the project**

The project used local models including qwen3:0.6b and qwen3:4b, which were the two main models used in the benchmark.

```
[10]: !ollama list

NAME                ID              SIZE      MODIFIED
qwen3:0.6b          7df6b6e09427    522 MB    3 seconds ago
moondream:latest    55fc3abd3867    1.7 GB    30 minutes ago
qwen3:4b            359d7dd4bcda    2.5 GB    5 hours ago
gemma3:4b           68ceab7e468a    3.4 GB    15 hours ago
gemma3:latest       68ceab7e468a    3.4 GB    15 hours ago
```

Figure 4:02_ Models

2. **What is model quantization?**

Model quantization is the process of representing model weights with fewer bits, such as 8-bit or 4-bit values instead of full-precision values. It reduces memory and storage requirements and makes local inference easier on modest hardware, although it may slightly reduce accuracy or output quality.

Part 4: Prompt Engineering

1. **Did the model produce different answers? Why?**

Yes. The model produced different answers because each prompt targeted a different audience and included different requirements. The model adapted its vocabulary, tone, examples, and level of detail to the instructions.

2. **Comparison of two DAG answers**

The first answer is formal, detailed, and structured. It includes a definition, an example, an adjacency matrix, a real-world application, and Python code. The second answer uses simpler vocabulary and a concrete cake-baking example for a first-year student.

- ✓ Role: “University professor” guides the model toward an educational and explanatory tone.
- ✓ Context: The target audience changes the language level: technical for the first answer and beginner-friendly for the second.
- ✓ Constraints: The first prompt requires five specific elements, while the second requires simple language and one concrete example.
- ✓ Output format: The required elements in the first prompt lead to a structured answer with sections, a table, and code; the second uses a simpler explanatory format.
- ✓ Specificity: More precise instructions produce a more targeted answer. However, the model did not fully respect conciseness in the second answer, showing that prompts guide output but do not guarantee perfect compliance.

Part 5: Model Management

1. **Difference between `ollama list` and `ollama ps`**

ollama list shows the models downloaded and stored on disk, while ollama ps shows the models currently loaded in memory and ready to generate responses. A downloaded model is installed but inactive, while a loaded model is active in RAM or VRAM.

Part 6: Comparing Two Local Models

1. **Benchmark questions:**

Which model is faster? Which produces better code? Which uses more memory? Which would you choose for programming assistance? Is the largest model necessarily the best?

The qwen3:0.6b model is faster. Its response time was 74.4 seconds, compared with 506.3 seconds for qwen3:4b.
Both models produced correct code: both passed all reference tests from -5 to 1999 and received 5/5 for code correctness. However, qwen3:4b produced a more detailed and better-explained answer, so it received the higher response-quality score.
The qwen3:4b model uses more memory because it has more parameters. The qwen3:0.6b model is smaller and requires fewer resources.
For quick programming assistance and limited hardware, I would choose qwen3:0.6b because it is much faster and still produced correct code. For more complex programming tasks, I would choose qwen3:4b because its explanations are more complete.
The largest model is not necessarily the best. The best model depends on the task, the required quality, the response time, and the available memory. In this experiment, qwen3:4b provided better explanations, but qwen3:0.6b offered much faster responses with the same code-correctness score.

2. **Benchmark results**

Metric	qwen3:0.6b	qwen3:4b
Disk size	0.52 GB	2.50 GB
Loaded size	1.02 GB	3.19 GB
VRAM	0.0 GB	0.0 GB
Load time	2.37 s	7.40 s
Response time	23.35 s	204.89 s
Tokens	531	1193
Tokens/s	28.37	6.14
System RAM used	67.7%	83.5%
Response quality /5	4	5
Code correctness /5	5	5

Table 1: Data of two local models

The results show that qwen3:0.6b is faster and lighter, while qwen3:4b consumes more memory and takes much longer to respond. For a computer with limited RAM, qwen3:0.6b is the better deployment choice because it loads faster, responds faster, and uses less memory.

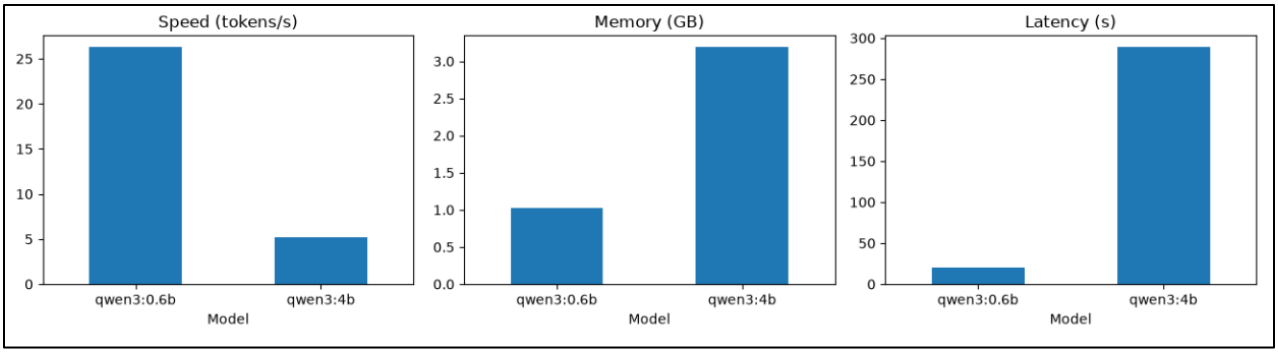


Figure 5:Comparison of the two local models.

Part 7: Building a Local AI Chatbot

1. **Stateless chatbot**

The first chatbot version treated every question independently and sent only the current question to Ollama. It could answer individual questions but could not use information from earlier messages.

```
[53]: # Non-interactive test (works without typing):
for q in ["What is machine learning?", "Explain reinforcement learning."]:
    print("You:", q); print("AI:", ask_ai(q)[:500], "...\n")
```

```
You: What is machine learning?
AI: Machine learning is a subset of artificial intelligence that involves training algorithms to learn patterns fr
om data and make decisions or predictions with minimal human intervention. The core idea is that machines can impr
ove their performance by learning from data, rather than being explicitly programmed to do so. Here's a breakdown:

1. **Basic Concepts**:
   - **Algorithms**: These are the rules or procedures the machine uses to process data.
   - **Models**: These are specific algorith ...

You: Explain reinforcement learning.
AI: Reinforcement Learning (RL) is a type of machine learning where an agent (often a learner or autonomous agent)
learns to make decisions by interacting with an environment. The goal is to maximize cumulative rewards through tr
ial and error, with the environment providing feedback in the form of rewards or penalties. Here's a breakdown:

1. **Agent and Environment**:
   The agent is an entity (e.g., a robot, player, or AI) that interacts with an environment (a world where actions
are taken and o ...
```

Figure 6: 03_Chatbot

Part 8: Conversation Memory

1. **Why does the model remember the name?**

The model appears to remember the name because the Python application stores the conversation history in the `history` variable and sends it again with each new request. The LLM itself is not permanently remembering the name, because nothing is stored inside the model after the request ends.

```
print("What the model actually received on the last call:\n" + "-"*50)
print("\n".join(history[:-1]) + "\nAssistant:")
```

```
What the model actually received on the last call:
--------------------------------------------------
User: My name is Ahmed.
Assistant: Ahmed, thank you for sharing your name! How can I help you today?
User: What is my name?
Assistant:
```

Figure 7: response of the model for conversation history

2. **Application architecture**

The final application follows this pipeline:
Browser -> Flask/Python -> conversation history plus document retriever -> Ollama -> local LLM.
The browser sends a question to Flask, Flask prepares the context, sends the request to Ollama, and returns the answer with sources, response time, and token statistics.

Part 9: Local AI for Software Engineering

1. **Programming assistance**

The models were tested with programming tasks such as writing code, reviewing Java code, identifying bugs, and correcting variable shadowing. Both models produced correct code, while the larger model generally provided more detailed explanations.

Part 10: Structured Output

1. **Why is structured output important?**

Structured output is important because it makes an LLM response predictable and easier for software to process. With a defined schema, the application receives required fields, valid data types, and more reliable values, which reduces parsing errors and makes validation, storage, and display easier.

Part 11: Local Document Assistant

1. **Document grounding**

The document assistant was instructed to answer only from the supplied document. For an out-of-scope question such as quantum computing, the correct answer was Information not available. The notebook also shows that grounding improves control over the answer but does not guarantee perfect compliance in every generation.

Part 12: Mini-RAG

1. **Definition of RAG:**

RAG means Retrieval-Augmented Generation. Before generating an answer, the application searches external documents and adds the most relevant text to the prompt. In this project, the retriever searches local course files such as ai.txt, course.txt, machine_learning.txt, reinforcement_learning.txt, and deep_learning.txt.

2. **RAG test results:**

Questions about reinforcement learning, deep learning, and supervised learning were answered using the retrieved course documents, while a question about world cup 2018 returned Information not available. Because no relevant document was found.

```
[49]: import importlib, rag
      importlib.reload(rag)

      retriever = rag.Retriever(rag.load_documents("documents"))

      def rag_answer(question, k=1, model=None):
          hits, ctx = retriever.context(question, k)
          out = generate(rag.build_prompt(question, ctx), model=model, think=False)
          return hits, out["response"], out["wall_s"]

      for q in ["What is reinforcement learning?",
                "Which hardware is needed to train neural networks with many layers?",
                "What is supervised learning?",
                "Who won the 2018 World Cup?"]:
          hits, ans, t = rag_answer(q)
          print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")

Q: What is reinforcement learning?
  retrieved: [('course.txt', 2.05727967535628)]
  A: Reinforcement learning learns through interaction with an environment using rewards and penalties.
  (2.5s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 10.176841870048676)]
  A: GPU.
  (2.2s)
```

Figure 8 : 05_RAG

Part 13: Multimodal AI

1. **Difference between an LLM and a multimodal LLM**

A traditional LLM mainly processes text, while a multimodal LLM can process several input types such as text and images. In the notebook's vision experiment, the multimodal model identified the image content as an airplane.

2. **Additional information the vision model receives**

The vision model receives the image itself in addition to the text prompt. It can analyze visual information such as objects, shapes, colors, surroundings, and visible text.

3. **Which applications could use local vision AI?**

Local vision AI can be used for document analysis, OCR, industrial inspection, education, medical-image preprocessing, and surveillance analysis. Running it locally can help keep sensitive images on the computer instead of sending them to a cloud service.



Figure 9:Results of multimodal LLM

Part 14: Final Flask Application

1. **Final application functionality**

The final Flask application runs locally, uses Ollama and a local LLM, provides a web interface, answers questions about the course, uses local documents as knowledge, maintains conversation history, supports structured responses, displays the answer, shows sources, and records response time and token statistics. The notebook test confirms that these requirements were implemented.

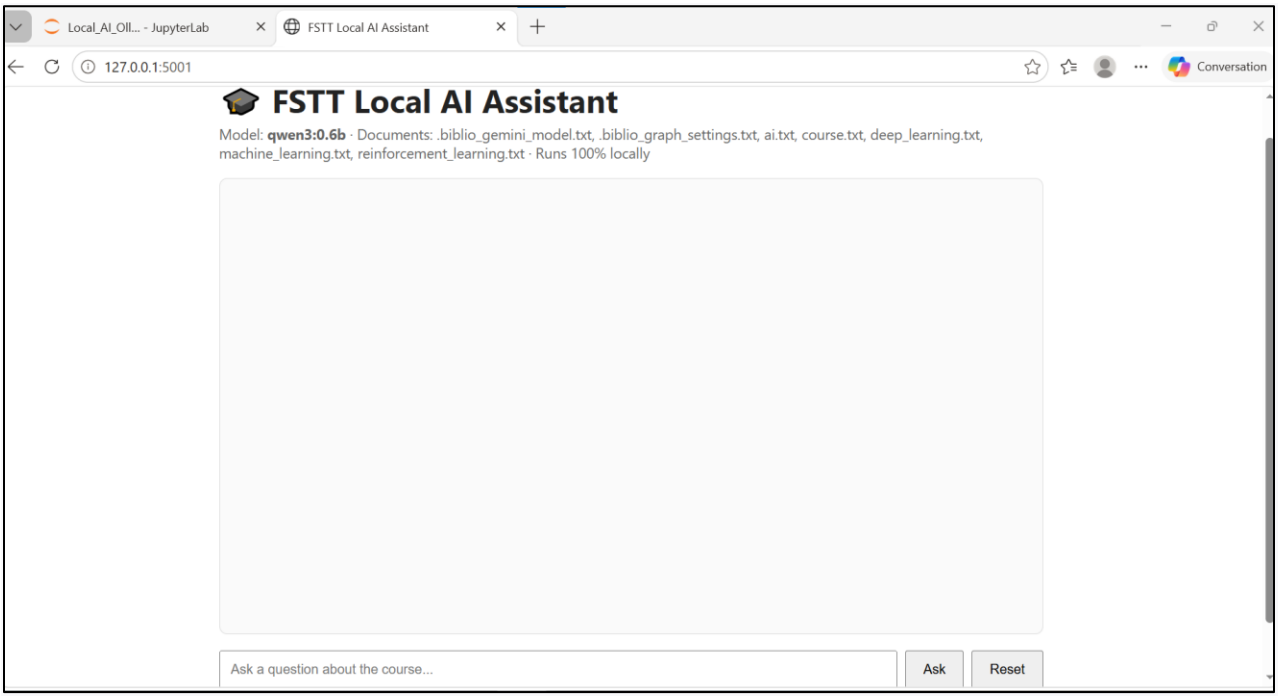


Figure 10: 06_Webapp

Part 15: Security Risks

1. **Risks when exposing Ollama to a network**

Ollama normally binds to 127.0.0.1:11434, and the local API does not require authentication by default. If the server is exposed to an untrusted network, unauthorized users could send prompts, consume CPU or RAM, access sensitive prompts or retrieved documents, or exploit malicious retrieved instructions. The safest configuration is to keep Ollama on localhost or protect it with firewall rules, authentication, a reverse proxy, and HTTPS/TLS.

Part 16: Final Questions

Below are the questions as mentioned in the notebook:

NB. They are already covered by the previous sections.

- **Q1. What is Local AI**?

Local AI is artificial intelligence that runs directly on the user's computer instead of sending prompts to a remote cloud service. In this project, Ollama runs the language model locally, so the questions and generated answers remain on the local machine

- **Q2. Difference between a Cloud LLM and a Local LLM?**

A cloud LLM runs on remote servers and is accessed through the Internet or an online API. A local LLM runs on the user's own computer. Cloud models usually provide more computing power and may be more capable, while local models provide better privacy, offline access, and no per-request cloud cost.

- **Q3. What role does Ollama play?**

Ollama manages and runs language models locally. It downloads the models, loads them into memory, starts the local inference service, and provides an API that Python and Flask can use to send prompts and receive responses. Ollama's local API is normally available at http://localhost:11434/api.

- **Q4. What is model quantization?**

Model quantization is the process of representing model weights with fewer bits, such as 8-bit or 4-bit values instead of full-precision values. This reduces the model's memory and storage requirements and can make local inference faster, although it may slightly reduce accuracy or output quality.

- **Q5. Why does model size affect memory consumption?**

A larger model contains more parameters, and those parameters must be loaded into RAM or VRAM during inference. Therefore, larger models generally require more memory. In this benchmark, qwen3:4b used 3.19 GB when loaded, compared with 1.02 GB for qwen3:0.6b.

- **Q6. Why can a smaller model sometimes be preferable?**

A smaller model can be preferable because it starts faster, responds faster, uses less memory, and works on less powerful hardware. It is also more suitable for interactive applications where low latency is important. In my test, qwen3:0.6b achieved 28.37 tokens/s, compared with 6.14 tokens/s for qwen3:4b.

- **Q7. Advantages of local inference?**

The main advantages are:
Better privacy because prompts and documents can remain on the computer.
Operation without an Internet connection.
No cloud API cost per token.
Full control over the selected model and data.
Easy integration with local Python applications.

- **Q8. Disadvantages of local inference?**

The main disadvantages are:
Local hardware limits the model size and speed.
Larger models can require significant RAM or VRAM.
Local models may be less capable than the best cloud models.
The user is responsible for installing, updating, and securing the system.
Inference can be slow on a CPU or on a computer with limited memory.

- **Q9. Explain the architecture of your Python application.**

The application uses Flask as the web interface and Python as the application logic. The browser sends a question to the Flask server. The application keeps recent conversation history, searches the local course documents with the retriever, and builds a prompt containing the relevant context. It then sends this prompt to Ollama, receives the local LLM response, and returns the answer, retrieved sources, response time, and token statistics to the browser.

- **Q10. Explain the role of the REST API.**

The REST API allows Python and Flask to communicate with Ollama through HTTP requests. For example, the application sends a POST request containing the model name, prompt, and options to Ollama's /api/generate or /api/chat endpoint. Ollama then returns a JSON response containing the generated text and performance metadata. Local requests do not require an authentication header by default.

- **Q11. What is RAG?**

RAG means Retrieval-Augmented Generation. Before generating an answer, the application searches external documents and adds the most relevant passages to the prompt. In this project, the retriever searches the local course files and Ollama generates an answer using the selected documents as context.

- **Q12. Difference between an LLM and a multimodal LLM?**

A traditional LLM mainly processes and generates text. A multimodal LLM can work with several types of input, such as text and images, and sometimes audio or video. For example, a multimodal model could analyze an uploaded diagram, while a text-only LLM could only analyze a textual description of that diagram.

- **Q13. Compare two local models experimentally.**

I compared qwen3:0.6b and qwen3:4b using the same prompt and the same computer. The smaller model was much faster and used less memory. The larger model generated more tokens, but it had a much longer response time. the measurements confirm the performance and resource differences, but they do not prove that qwen3:4b gives better answers. (check Table 1).

- **Q14. Which model would you deploy on a computer with limited RAM? Explain.**

I would deploy qwen3:0.6b on a computer with limited RAM. It used only 1.02 GB when loaded, compared with 3.19 GB for qwen3:4b, and it used less system RAM overall: 67.7% versus 83.5%. It was also much faster, reaching 28.37 tokens/s instead of 6.14 tokens/s. The larger model would only be preferable if its additional quality justified its higher memory usage and much longer response time.

- **Q15. What security risks exist when exposing a local LLM server to a network?**

The main risks are unauthorized use of the computer's resources, exposure of private prompts or documents, model management by unauthorized users, and prompt injection through retrieved files. Ollama's local API does not require authentication by default, so it should not be exposed directly to an untrusted network. The server should remain on localhost or be protected with firewall rules, authentication, and HTTPS/TLS through a reverse proxy.

Part 17: Final Mini Project

1. **Final system architecture**

The final system is organized as follows:

```
Browser -> Flask/Python
          ├── Conversation history
          └── Local documents -> Retriever
                      ↓
                  Ollama
                    ↓
                  Local LLM
```

2. **Final project structure**

```
LocalAI_Lab/
├── source/
│   ├── app.py
│   ├── chatbot.py
│   ├── rag.py
│   ├── benchmark.py
│   └── templates/
│       └── index.html
├── documents/
├── screenshots/
├── report.pdf
└── README.md
```

Part 18: Deliverables

The final Part 19 check in the notebook shows that every required path exists before submission.

```
for p in required:
    print("✅" if os.path.exists(p) else "⬜ missing", p)
print("\nDocuments:", sorted(os.listdir("LocalAI_Lab/documents")))
```

```
••• ✅ LocalAI_Lab/report.pdf
    ✅ LocalAI_Lab/README.md
    ✅ LocalAI_Lab/source/app.py
    ✅ LocalAI_Lab/source/chatbot.py
    ✅ LocalAI_Lab/source/rag.py
    ✅ LocalAI_Lab/source/benchmark.py
    ✅ LocalAI_Lab/screenshots/01_installation.png
    ✅ LocalAI_Lab/screenshots/02_model.png
    ✅ LocalAI_Lab/screenshots/03_chatbot.png
    ✅ LocalAI_Lab/screenshots/04_python.png
    ✅ LocalAI_Lab/screenshots/05_rag.png
    ✅ LocalAI_Lab/screenshots/06_webapp.png
```

Figure 11:check list of delivrable

Conclusion

This project demonstrates the construction of a fully local AI assistant using Ollama, Python, Flask, conversation history, local document retrieval, structured responses, and performance monitoring. The benchmark shows that qwen3:0.6b is the most practical model for the tested computer because it offers much better response speed and lower memory usage than qwen3:4b. The final application satisfies the project requirements and the final verification cell confirms that all required files exist.